

21. Алгоритми за намиране на минимални покриващи дървета и най-кратки пътища в свързани графи.

1. Въведение

Голяма част от практическите задачи върху мрежи - проектиране на комуникационни и електропреносни мрежи, маршрутизация в компютърни мрежи, планиране на транспорт - се свеждат до два фундаментални класа задачи върху **свързан претеглен граф**:

1. **Минимално покриващо дърво (MST)** - да свържем всички върхове с минимален общ разход, без цикли.
2. Най-кратки пътища (Shortest Paths) - да намерим път с минимална сума от тегла между върховете.

И двата класа задачи се решават с помощта на алчни (greedy) алгоритми или динамични подходи, чиято коректност почива на общи структурни особености на графите - свойството на разреза при MST и оптималната подструктура при най-кратките пътища.

2. Основни дефиниции

Нека $G = (V, E)$ е граф с множество върхове V и множество ребра E , върху който е зададена тегловна функция $w : E \rightarrow \mathbb{R}$. Означаваме $|V| = n$ и $|E| = m$.

Свързан граф - G е свързан, ако между всеки 2 върха съществува път. За неориентиран свързан граф важи $m \geq n - 1$.

Дърво - Дърво е свързан ацикличен неориентиран граф. Всяко дърво с n върха има точно $n - 1$ ребра; добавянето на кое да е ребро създава точно един цикъл, а премахването на кое да е ребро го прави несвързан.

Покриващо дърво (spanning tree). Подграф $T = (V, E_T)$ на свързан граф G , който е дърво и съдържа всички върхове на G . Теглото му е:

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

Минимално покриващо дърво. Покриващо дърво с минимално възможно тегло. То не е задължително единствено (единствено е тогава и само тогава, когато всички тегла на ребрата са различни).

Тегло на път и най-кратък път. За ориентиран или неориентиран претеглен граф, теглото на път $p = \langle v_0, v_1, \dots, v_k \rangle$ е $w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$. Разстоянието от u до v е:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \xrightarrow{p} v\}, & \text{ако съществува път,} \\ \infty, & \text{иначе.} \end{cases}$$

Най-кратък път е път p , за който $w(p) = \delta(u, v)$.

3. Минимални покриващи дървета

3.1 Обобщен алчен алгоритъм

Всички класически алгоритми за MST са частни случаи на един **обобщен алчен алгоритъм**, който поддържа подмножество ребра, инвариант на което е, че A е подмножество на някое MST. На всяка стъпка се добавя безопасно ребро - ребро (u, v) , за което $A \cup \{(u, v)\}$ отново е подмножество на някое MST.

GENERIC-MST(G, w):

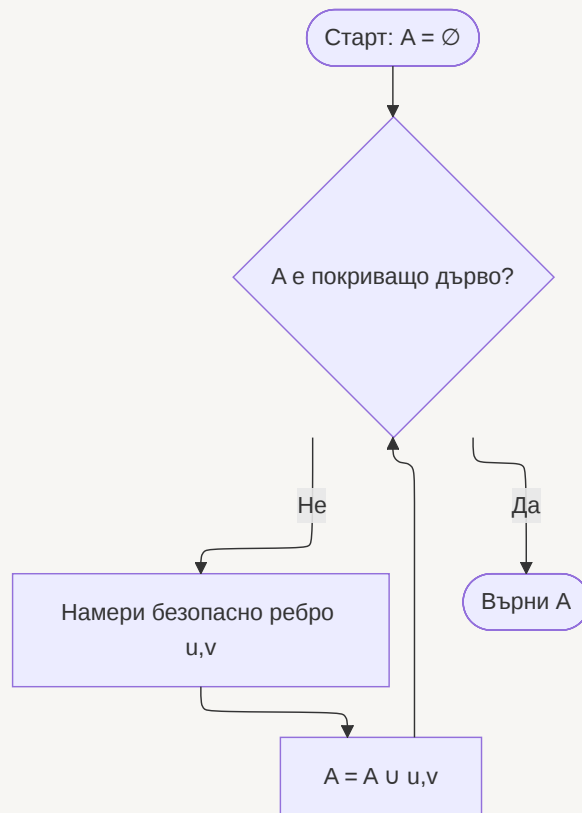
$A = \emptyset$

```

while A не образува покриващо дърво:
    намери ребро  $(u, v)$ , безопасно за A (A продължава да
    бъде MST)
     $A = A \cup \{(u, v)\}$ 

return A

```



3.2 Свойство на разреза (cut property)

Основен фундамент за алгоритмите за MST е Свойството на разреза.

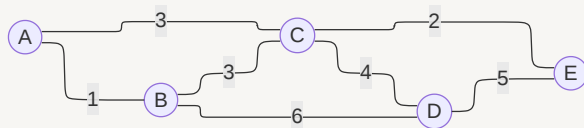
Разрез (cut) на граф G е разделяне на множеството V на две непресичащи се подмножества S и $V \setminus S$. Ребро $(u, v) \in E$ пресича разреза, ако $u \in S$ и $v \in V \setminus S$.

Теорема (Свойство на разреза): За всеки разрез на графа, реброто с най-малко тегло (light edge), което пресича разреза, принадлежи на поне едно

минимално покриващо дърво на G .

3.3 Пример

Разглеждаме неориентирания претеглен граф:



Сортирани по тегло ребра:

$AB(1)$, $CE(2)$, $AC(3)$, $BC(3)$, $CD(4)$, $DE(5)$, $BD(6)$.

МПД: $\{AB, CE, AC, CD\}$ с общо тегло $1 + 2 + 3 + 4 = 10$.

3.4 Алгоритъм на Крускал

Алгоритъмът на Крускал изгражда MST чрез постепенно добавяне на ребра към първоначално празно множество (гора от отделни върхове).

Логика на алгоритъма:

1. Сортира всички ребра в E във възходящ ред според тяхното тегло.
2. Инициализира гора, в която всеки връх е отделно дърво.
3. Обхожда сортираните ребра. За всяко ребро (u, v) :
 - а. Ако u и v принадлежат на различни дървета, реброто се добавя към MST, а двете дървета се обединяват.
 - б. Ако принадлежат на едно и също дърво, добавянето му би създавало цикъл, затова реброто се отхвърля.

Структура от данни (Union-find)

За проверката дали 2 върха са в едно и също дърво се използва структурата Система от непресичащи се множества.

```
MST-KRUSKAL( $G, w$ ):
```

```
   $A = \emptyset$ 
```

```
  for всеки връх  $v \in V$ : MAKE-SET( $v$ )
```

```
  сортирай ребрата  $E$  по нарастващо тегло  $w$ 
```

```
  for всяко ребро  $(u, v) \in E$  по нарастващ ред:
```

```
    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ):
```

```
       $A = A \cup \{(u, v)\}$ 
```

```
      UNION( $u, v$ )
```

```
  return  $A$ 
```

Сложност. Сортирането на ребрата е $O(m \log m)$. Общата времева сложност е $O(E \log V)$.

3.5 Алгоритъм на Прим

За разлика от Крускал, алгоритъмът на Прим изгражда MST като едно единствено дърво, което расте от произволен начален връх.

Логика на алгоритъма:

1. Избира се произволен стартов връх r и се добавя към дървото A .
2. На всяка стъпка се намира реброто (u, v) с минимално тегло, такова че $u \in A$, а $v \notin A$ (ребро, пресичащо разреза между дървото и останалите върхове).
3. Връх v и реброто (u, v) се добавят към дървото A .
4. Процесът се повтаря, докато всички върхове попаднат в A .

Структура от данни (Priority Queue / Min-Heap):

За бързо намиране на най-лекото ребро се използва приоритетна опашка (Min-Heap). В нея се съхраняват върховете извън дървото, като ключът на всеки връх е минималното тегло на ребро, свързващо го с дървото.

```

MST-PRIM( $G, w, r$ ):
  for всеки връх  $u \in V$ :  $key[u] = \infty$ ;  $\pi[u] = NIL$ 

   $key[r] = 0$ 

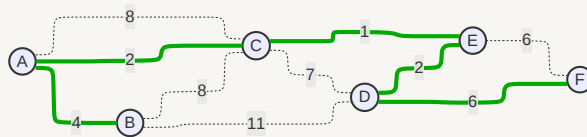
   $Q = \text{min-приоритетна опашка с всички върхове}$ 

  while  $Q \neq \emptyset$ :
     $u = \text{EXTRACT-MIN}(Q)$ 
    for всеки съсед  $v$  на  $u$ :
      if  $v \in Q$  и  $w(u, v) < key[v]$ :
         $\pi[v] = u$ 
         $key[v] = w(u, v)$  // DECREASE-KEY
  return  $\{(v, \pi[v]) : v \in V \setminus \{r\}\}$ 

```

Сложност според реализацията на опашката:

Структура на опашката	Общо време
Масив	$O(n^2)$
Двоична пирамида	$O(m \log n)$
Пирамида на Фибоначи	$O(m + n \log n)$



4. Най-кратки пътища

4.1 Оптимална подструктура и релаксация

Разглеждаме предимно **ориентирани** претеглени графи (неориентираните са частен случай). Класически варианти на задачата:

- От един източник (single-source) - $\delta(s, v)$ за всички v .

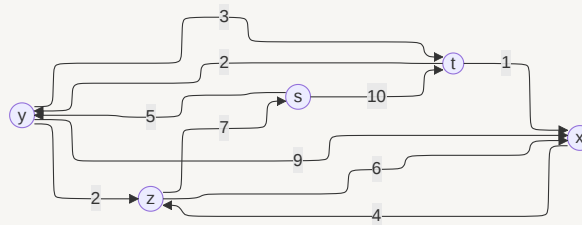
- Всички двойки (all-pairs) - $\delta(u, v)$ за всяка двойка.

Оптимална подструктура: Всеки подпът на най-кратък път е също най-кратък път.

Отрицателни цикли. Ако от s е достижим цикъл с отрицателно тегло, разстоянията до върховете по него не са дефинирани ($-\infty$). Алгоритмите или изискват липса на такива цикли, или ги откриват.

Релаксация е ключовата елементарна операция. За всеки връх v поддържаеме атрибут $d[v]$, който е горна граница на теглото на най-краткия път от източника s до v . Операцията `Relax( $u, v, w$ )` проверява дали можем да подобрим намерения до момента път до v , като минем през u : Ако $d[v] > d[u] + w(u, v) \implies d[v] = d[u] + w(u, v)$, $\pi[v] = u$, където $\pi[v]$ е предшественикът на v в дървото на най-кратките пътища.

4.2 Пример (ориентиран граф)



Най-кратки разстояния от s :

$$d[s] = 0, \quad d[y] = 5, \quad d[z] = 7, \quad d[t] = 8, \quad d[x] = 9,$$

напр. $s \rightarrow y \rightarrow t \rightarrow x$ с тегло $5 + 3 + 1 = 9$.

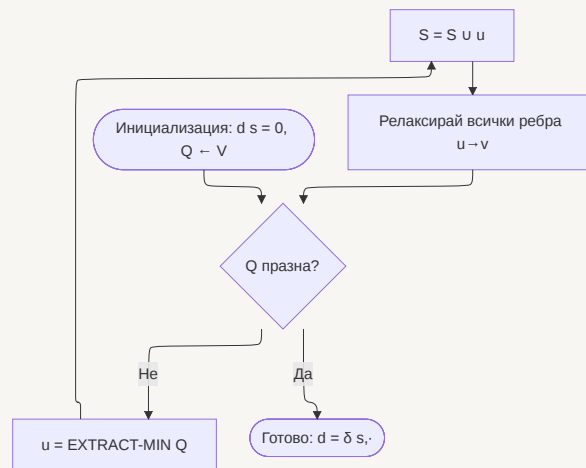
4.3 Алгоритъм на Дейкстра

Алгоритъмът на Дейкстра решава задачата за най-кратък път от един източник за графи, в които няма ребра с отрицателно тегло ($w(u, v) \geq 0 \forall (u, v) \in E$).

Логика:

1. Инициализира $d[s] = 0$ и $d[v] = \infty$ за всички останали върхове.
2. Поддържа множество S от върхове, чиито най-кратки пътища вече са окончателно намерени. Първоначално $S = \emptyset$.
3. Използва приоритетна опашка Q , съдържаща всички върхове от $V \setminus S$, с ключ техните d -стойности.
4. Докато Q не е празна:
 - a. Извлича върха $u \in Q$ с минимално $d[u]$ и го добавя към S .
 - b. Релаксира всички изходящи ребра от u . Ако за някое ребро (u, v) стойността $d[v]$ се намали, позицията на v в Q се актуализира (Decrease-Key).

```
DIJKSTRA( $G, w, s$ ):  
  INITIALIZE-SINGLE-SOURCE( $G, s$ )  
   $S = \emptyset$   
   $Q = \text{min-приоритетна опашка с всички върхове по ключ } d[\cdot]$   
  
  while  $Q \neq \emptyset$ :  
     $u = \text{EXTRACT-MIN}(Q)$   
     $S = S \cup \{u\}$   
    for всяко ребро  $(u, v)$ :  
      RELAX( $u, v, w$ )           // при подобрене: DECREASE-KEY( $v$ )
```



Коректност:

Алгоритъмът на Дейкстра работи поради факта, че всички тегла са неотрицателни. Когато връх u бъде извлечен от опашката Q , неговото разстояние $d[u]$ вече е равно на истинското най-кратко разстояние $\delta(s, u)$. Никой бъдещ път, минаващ през върхове, които все още са в Q , не може да бъде по-къс, защото всяко допълнително ребро би добавило неотрицателно тегло към вече по-голямо или равно начално разстояние.

Сложност:

Структура	Време
Масив	$O(n^2)$
Двоична пирамида	$O((n + m) \log n)$
Пирамида на Фибоначи	$O(n \log n + m)$

4.4 Алгоритъм на Белман–Форд

Когато графът съдържа ребра с отрицателни тегла, алгоритъмът на Дейкстра е неприложим. Алгоритъмът на Белман–Форд решава този проблем, като освен това може да открие наличието на отрицателни цикли (цикли, чиято сума от теглата е < 0). Ако съществува такъв цикъл, достижим от източника, то най-кратък път математически не съществува (можем да се въртим в цикъла до безкрайност и разстоянието да клони към $-\infty$).

Логика:

Най-кратък път в граф без отрицателни цикли може да съдържа максимум $|V| - 1$ ребра.

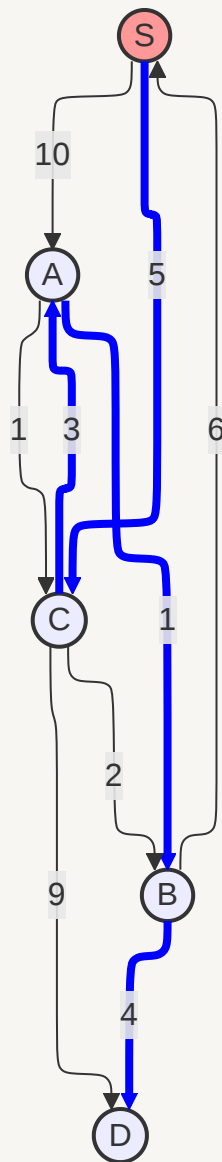
1. Инициализира $d[s] = 0$ и $d[v] = \infty$.
2. Прави $|V| - 1$ итерации. На всяка итерация се обхождат в графа и се прилага операцията `Relax(u, v, w)`.
3. След $|V| - 1$ итерации, ако графът няма отрицателни цикли, всички $d[v]$ съдържат минималните пътища ($\delta(s, v)$).
4. (Проверка за отрицателни цикли): Прави се още едно обхождане на всички ребра. Ако за някое ребро е изпълнено $d[v] > d[u] + w(u, v)$, това означава, че съществува отрицателен цикъл.

```
BELLMAN-FORD(G, w, s):  
    INITIALIZE-SINGLE-SOURCE(G, s)  
  
    for i = 1 to |V| - 1:  
        for всяко ребро (u, v) ∈ E:  
            RELAX(u, v, w)  
  
    for всяко ребро (u, v) ∈ E:                // проверка за отриц  
    ателен цикъл  
        if d[v] > d[u] + w(u, v):  
            return FALSE  
    return TRUE
```

Сложност:

Алгоритъмът извършва $O(E)$ операции за всяка от V итерации.

Следователно времевата му сложност е строго $O(V \cdot E)$. Въпреки че е по-бавен от Дейкстра, той е много по-универсален.



4.5 Всички двойки: алгоритъм на Флойд-Уоршал

За задачата “всички двойки” ефективен е **динамичният** алгоритъм на Флойд-Уоршал. Допуска отрицателни тегла (без отрицателни цикли).

Нека $d_{ij}^{(k)}$ е теглото на най-краткия път от i до j , при който всички междинни върхове са от множеството $\{1, 2, \dots, k\}$. Рекурентната зависимост е:

$$d_{ij}^{(k)} = \begin{cases} w(i, j), & k = 0 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}), & k \geq 1. \end{cases}$$

Сложността му е изключително предвидима – $O(n^3)$, благодарение на три вложени цикъла, което го прави отличен избор за плътни графи.

```
FLOYD-WARSHALL(W):           // W е матрица n×n на теглата
    D = W
    for k = 1 to n:
        for i = 1 to n:
            for j = 1 to n:
                if D[i][k] + D[k][j] < D[i][j]:
                    D[i][j] = D[i][k] + D[k][j]
    return D
```

Сложност: време $O(n^3)$, памет $O(n^2)$.

4.6 Сравнение

Алгоритъм	Задача	Отриц. тегла	Време
Дейкстра	1 източник	не	$O(m+n \log n)$ (Фибоначи)
Белман–Форд	1 източник	да (+ откриване на цикъл)	$O(nm)$
Фloyd–Уоршал	всички двойки	да (без отриц. цикли)	$O(n^3)$